

Forge, and extending SPRFST

Forge is the package manager. It is built into the `sprfst` command, needs no account and no online registry, and keeps every dependency inside your project where you can read it.

A project

```
project.sprfst  what this is, and what it depends on
forge.lock     exactly what was installed, with checksums
src/           your code
tests/         your tests
assets/        files your program loads
packages/      dependencies, copied in
build/         output, safe to delete
```

`project.sprfst` is deliberately dull:

```
[package]
name = "notes"
version = "0.1.0"
entry = "src/main.spf"
description = "A small notebook"

[packages]
mathx = path:../mathx
palette = git:https://example.com/palette.git
```

The commands

Command	What happens
<code>sprfst add <name> --path=../somewhere</code>	copy a local package in and record it
<code>sprfst add <name> --git=<url></code>	clone a package in and record it
<code>sprfst remove <name></code>	delete it from packages/ and both files
<code>sprfst install</code>	make packages/ match project.sprfst
<code>sprfst package</code>	build a .forge archive of this project

Adding a package prints its digest:

```
$ sprfst add mathx --path=../mathx
  installed mathx          45c9c4769351
```

and writes `forge.lock` :

```
# Forge lockfile — do not edit by hand
mathx path:../mathx 45c9c4769351aebc70c04263a9920aaa3923f635402bf8ce76ac519cd17a0b7b
```

That digest is a SHA-256 over every file in the package. On the next `sprfst install`, the contents are hashed again and compared. If they differ, the install **stops** rather than quietly using something else. Commit `forge.lock`; it is what makes a build the same for everybody.

Using a package

A package is just a project with modules in it. Import it by name:

```
use std.io
use mathx

fn main() {
    io.say("{mathx.double(21)}")
}
```

The import is resolved in this order:

1. a file beside the one importing it — `shapes.spf`, or `shapes/mod.spf`
2. the entry point named in `packages/<name>/project.sprfst`
3. `packages/<name>/src/<name>.spf`, then `src/main.spf`, then `<name>.spf`
4. the standard library in `$SPRFST_HOME/std/`

So a local file always wins over a package, and a package always wins over nothing. Nothing is fetched behind your back while you are compiling.

Publishing

```
$ sprfst package
packaged ./notes-0.1.0.forge 2 files, 0 KB, 77730369e687
```

A `.forge` file is a plain archive: a `FORGE1` header, one `file <path> <length>` record per file, the bytes, and a SHA-256 of the whole body. Hand it over however you like —a web server, a shared folder, an email. There is no registry to sign up to, and `--git=` means a repository is already a package.

Extending the language

Three doors, from the easiest to the deepest.

1. SPRFST modules

The standard library's own `core`, `collections`, `testing` and `ai` modules are ordinary SPRFST source in `std/`. Anything you write can sit beside them, and `use std.yourthing` will find it. That is the whole extension mechanism for most purposes.

2. Shelling out

`sys.run` gives you the rest of the operating system, and `std.json` makes the conversation easy:

```
use std.io
use std.sys
use std.json
```

```
fn main() {
  let listing = sys.run("ls -1").trim().lines()
  io.say("{listing.len()} entries here")
  io.say(json.write(["cwd_entries": listing.len(), "platform": sys.platform()]))
}
```

3. Native functions

The native table in `compiler/include/sprfst/natives.h` is one line per function:

```
X(MATH_SQRT, "math", "sqrt", "(Num) -> Num", "Square root")
```

The line declares the module, the name, the signature the typechecker enforces and the help text that `sprfst docs` and Studio show. The implementation goes in `compiler/src/nativelib.c` under the matching `NF_` case. Rebuild, and `math.sqrt` exists everywhere—in the checker, the editor's autocomplete, the documentation and the runtime—with no glue code, no bindings file and no registration call.

`extern fn` declarations reserve the same mechanism for functions provided by a host application embedding the runtime.

Editors other than Studio

`sprfst studio` starts a line based service: one JSON request per line, one JSON reply. It answers `check`, `outline`, `tokens`, `complete`, `hover`, `define`, `rename`, `format`, `natives` and `version`. Studio speaks it, and so can any editor you like—it is a dozen lines of glue in most of them.

```
$ echo '{"id":1,"op":"check","file":"src/main.spf"}' | sprfst studio
```

Security, briefly

!22 Packages are copied in, never run during installation. There are no install scripts.

!22 Everything is hashed, and a mismatch stops the build.

!22 `sprfst package` hashes the archive body as well.

!22 The runtime does nothing surprising: no network, no file writes and no process launches unless your program asks for them.

Exercise

Make a package of your own: `sprfst new palette`, give it a module with two or three colour helpers, then `sprfst add palette --path=../palette` from another project and use it. Change one character in the package and watch `sprfst install` refuse.